algorithms 2 homework 3

mahmood

April 28, 2023

Question 1.

Answer 1.0.1. my idea is to simply replace the usual weight function in the relax function of the bellman-ford algorithm call with an external time dependant weight function

```
RELAX(G,u,v):
    time_passed ← d[u]
    actual_weight ← GOOGLE-UPDATED-WEIGHT(time_passed, (u, v))
    if d[v] > d[u] + actual_weight then
        d[v] ← d[u] + actual_weight
        p[v] ← u
```

Question 2. modify the so that it substitutes $d[v] = -\infty$ for each vertex for which there exists a negative cycle over some path from the origin vertex to v
show why the algorithm is correct

Answer 2.0.1. im actually not sure of this solution

```
BELLMAN-FORD-OP(G,s):
    for i ← 1 to |V| − 1 do
        for (u, v) ∈ E do
            RELAX(G,u,v)
    found = false
    for (u, v) ∈ E do
        if d[v] > d[u] + w(u, v) then
            d[v] = −∞
            found = true
    return found
```

time complexity of the algorithm isnt affected, for s, the of the algorithm remains $\Theta\left(|V|^3\right)$ while for sparse graphs its $\Theta(|V| \cdot |E|)$

Question 3. given an undirected non-negatively weighted graph $G = (V, E, w)$ that describes the network of n computers and the direct connections between them. the weight of an edge (x, y) is the transition time of a message from x to y in their direct connection. given a vertex $m \in V$ that represents the network archive. write an efficient algorithm that finds the transition timing matrix of messages in the given network such that each message has to go through the computer m, i.e. it has to find the matrix $A[n \times n]$ such that $A[i, j]$ is the distance of the shortest path in the network from i to j that goes through m

Answer 3.0.1.

```
DISTANCE-MATRIX(G,m):
    initialize matrix mymatrix of dimensions |V| × |V|
    for machine_1 ∈ V do
        for machine_2 ∈ V do
            d_1 ← DISJKTRA(G, machine_1)
            d_2 ← DISJKTRA(G, machine_2)
            mydistance ← d_1[m] + d_2[m]
            mymatrix[machine_1, machine_2] = mydistance
    return mymatrix
```

Question 4. propose an algorithm that finds the distances from all vertices to a given vertex s in an undirected graph whose weights belong to the set $\{1, 2, 3, 4, 5\}$ in $\Theta(|E|)$ time

Answer 4.0.1. my idea is to turn each \$n\$-weighted edge into n vertices, so the distance remains the same but the graph is now all edges of weight 1, so we can just use the BFS algorithm which runs in $\Theta(|E|)$ time, although by doing this we are increasing the amount of edges in the graph by up to $5|E|$ but $\Theta(5|E|) = \Theta(|E|)$ so it doesnt matter
we should probably get rid of the new nodes we added or even better clone the graph and manipulate that one instead of modifying the original graph
im sure there is a better approach... but this is what i could come up with

```
MY-STUPID-ALGORITHM(G,s):
    for (u,v) ∈ E do
        construct w(u,v) − 1 new nodes and w(u,v) new edges of weight
          1 to connect between the new nodes and place the nodes
          between u and v
    return BFS(G,s)
```

the time complexity is $\Theta(|E|)$, the algorithm works because the algorithm BFS works on unweighted graphs and we basically modified the graph to have weights of only 1 and so it can be referred to as an unweighted graph

Question 5. given a directed, weighted graph $G = (V, E, W)$ and a vertex $v \in V$, we denote the length of the shortest path that starts at some vertex and ends at v by $\varphi(v) = \min\{\delta(u, v) \mid u \in V\}$
note that if the weights of the graph are non-negative then $\varphi(v) = 0$ for every v
write an algorithm that receives a graph with no negative cycles and finds $\varphi(v)$ for every vertex v in the graph

```
MY-OTHER-STUPID-ALG(G):
    initialize map φ of size |V|
    for v ∈ V do
        d ← BELLMAN-FORD(G,v)
        φ[v] ← ∞
        for u ∈ V do
            φ[v] ← min{φ[v], d[u]}
    return φ
```